SAECOIN — WALLET SECURITY DESIGN

Document: Wallet_Security_Design.md
Version: 1.0
Project: SAECOIN
Ticker: SAC (Provisional)
Status: Draft / Planning
Wallet Platforms: Android and Windows
Transaction Model: UTXO

---

1. PURPOSE

This document defines the security architecture for SAECOIN wallets.

Its primary objectives are:

- Protect users' private keys.
- Protect wallet recovery phrases.
- Prevent unauthorized transactions.
- Provide secure wallet backup.
- Support wallet recovery.
- Protect transaction signing.
- Minimize exposure of sensitive information.
- Provide consistent security across Android and Windows wallets.

Wallet security is a critical part of SAECOIN.

A blockchain transaction signed with a compromised private key may result in irreversible loss of funds.

---

2. WALLET SECURITY PRINCIPLES

SAECOIN wallets must follow these principles:

1. Users control their private keys.
2. Private keys must never be transmitted to remote servers.
3. Transaction signing must happen locally.
4. Recovery information must remain under user control.
5. Wallet data must be encrypted at rest.
6. Sensitive information must not appear in application logs.
7. Wallet backups must be recoverable.
8. Invalid transactions must be rejected before signing.
9. Remote nodes must not be trusted with private signing authority.
10. Security must take priority over convenience.

---

3. WALLET ARCHITECTURE

The proposed wallet architecture has four major components:

SAECOIN WALLET
|
|-- Wallet Interface
|
|-- Wallet Core
| |
| |-- Key Management
| |-- Address Generation
| |-- Transaction Construction
| |-- Transaction Signing
| |-- Balance Calculation
|
|-- Encrypted Wallet Storage
|
|-- Node/API Connection

The Android and Windows wallets should share the same underlying wallet-core security rules.

A shared Rust wallet core may be used where practical.

The platform interfaces will be responsible for secure interaction with their respective operating systems.

---

4. PRIVATE KEY MANAGEMENT

A private key controls the ability to authorize transactions from associated addresses.

Requirements:

- Private keys must be generated using a cryptographically secure random number generator.
- Weak random number generators must never be used.
- Private keys must never be hardcoded.
- Private keys must never be uploaded to a server.
- Private keys must never be included in analytics.
- Private keys must never be printed in application logs.
- Private keys must not be exposed through ordinary API responses.

The final key-generation algorithm and elliptic-curve selection remain TO BE DECIDED.

They must be selected through a formal cryptographic review.

---

5. SEED PHRASE DESIGN

SAECOIN may support mnemonic recovery phrases.

A seed phrase can allow users to restore their wallet.

The following decisions must be finalized:

- Mnemonic standard
- Word-list language
- Entropy size
- Checksum rules
- Seed derivation method
- Key derivation path
- Address derivation rules
- Optional passphrase behavior

BIP39 may be evaluated as a candidate standard, but it must not be adopted automatically without reviewing compatibility and cryptographic requirements.

The final recovery format must be documented before public wallet release.

---

6. WALLET CREATION

Proposed wallet creation flow:

1. User selects Create Wallet.
2. Application generates cryptographically secure entropy.
3. Wallet derives its recovery material using the approved standard.
4. Application displays the recovery phrase securely.
5. User is asked to verify selected words.
6. Wallet derives the required keys.
7. Wallet encrypts local key material.
8. Wallet creates the initial wallet database.
9. Wallet displays the receiving address.

The application must clearly warn users that anyone with the recovery phrase may be able to control their funds.

The recovery phrase must not be displayed again automatically after initial setup.

---

7. WALLET ENCRYPTION

Sensitive wallet data must be encrypted at rest.

Protected information may include:

- Private keys
- Seed material
- Key derivation information
- Wallet metadata that could expose sensitive information

The final encryption design must specify:

- Encryption algorithm
- Key derivation function
- Salt generation
- Nonce or initialization-vector requirements
- Authentication mechanism
- Password verification behavior
- Encryption format version

The encryption design must use reviewed cryptographic libraries.

Custom cryptographic algorithms must not be invented for wallet encryption.

A password alone must not be treated as a replacement for a secure recovery phrase.

---

8. PASSWORD AND UNLOCK POLICY

Wallets should support a local password or passphrase.

The password will be used to unlock protected wallet data.

Requirements:

- Passwords must never be stored as plaintext.
- Password-based key derivation must use an appropriate password-hardening function.
- Unlock attempts should be rate-limited where appropriate.
- Sensitive data should be cleared from memory when practical.
- Wallets should support manual locking.
- Wallets should support automatic locking after inactivity.

The final password derivation parameters must be benchmarked on supported devices.

---

9. ANDROID WALLET SECURITY

The Android wallet will use Kotlin and Jetpack Compose.

Security requirements:

- Use Android Keystore where appropriate.
- Protect sensitive application data.
- Avoid storing plaintext private keys.
- Avoid logging recovery information.
- Prevent sensitive information from appearing in notifications.
- Use secure application storage.
- Require authentication before sensitive operations.
- Support wallet locking.
- Consider biometric authentication as an optional convenience feature.

Biometric authentication must not replace the underlying cryptographic security model.

Android backup behavior must be configured carefully so encrypted wallet files and sensitive data are not unintentionally copied to unsecured locations.

The application must also protect against accidental exposure through screenshots and app-switcher previews where practical.

---

10. WINDOWS WALLET SECURITY

The Windows wallet will use the proposed Rust and Tauri architecture.

Security requirements:

- Keep private-key operations inside the trusted wallet core.
- Minimize communication between the frontend and privileged backend.
- Validate every frontend command.
- Restrict unnecessary Tauri capabilities.
- Protect local wallet files.
- Use operating-system security facilities where appropriate.
- Require wallet unlocking before signing.
- Avoid exposing private keys to JavaScript.
- Avoid storing passwords in configuration files.

Windows Credential Manager or other suitable Windows security facilities may be evaluated for protecting local encryption keys.

The final implementation must be reviewed for frontend-to-backend privilege escalation risks.

---

11. TRANSACTION SIGNING

Transaction signing must happen locally.

Proposed signing flow:

1. Wallet receives transaction information.
2. Wallet validates transaction structure.
3. Wallet checks available UTXOs.
4. Wallet calculates required fees.
5. Wallet displays recipient and amount.
6. User confirms the transaction.
7. Wallet unlocks the required signing key.
8. Wallet signs the transaction locally.
9. Wallet verifies the resulting signature.
10. Wallet broadcasts the signed transaction through a node.

A remote API must never be given direct access to the user's private signing keys.

The signing algorithm and signature serialization must match the finalized blockchain protocol.

---

12. TRANSACTION CONFIRMATION SECURITY

Before signing, the wallet must clearly display:

- Recipient address
- Transaction amount
- Transaction fee
- Total amount deducted
- Change output, where applicable
- Network type

The wallet must identify the active network clearly.

The user must be warned if:

- The recipient address is invalid.
- The transaction uses an unexpected network.
- The fee is unusually high.
- The available balance is insufficient.

- The transaction cannot be validated.

The wallet must not silently change the recipient or transaction amount after user confirmation.

---

13. WALLET BACKUP

Users must be able to back up their wallets.

Proposed backup options:

1. Recovery phrase backup
2. Encrypted wallet-file backup
3. Manual export of encrypted wallet data

Requirements:

- Explain backup security clearly.
- Warn against sharing backup files.
- Warn against storing recovery phrases in cloud notes or screenshots.
- Support backup restoration testing.
- Include backup format version information.
- Detect unsupported backup formats.

Wallet backups should not contain unnecessary unencrypted secrets.

---

14. WALLET RECOVERY

Proposed recovery process:

1. User selects Restore Wallet.
2. User enters the recovery phrase or imports an encrypted backup.
3. Application validates the recovery format.
4. Wallet derives the expected keys.
5. Wallet scans or synchronizes the relevant blockchain history.
6. Wallet reconstructs its transaction history.
7. Wallet calculates balances from verified UTXOs.
8. User sets a new local wallet password.
9. Wallet saves encrypted data.

Recovery must work without depending on a centralized account provider.

A wallet restore must not create a new unrelated wallet by mistake.

---

15. ADDRESS SECURITY

Wallet addresses must be generated using the finalized address-derivation system.

Requirements:

- Address encoding must include error detection.
- Mainnet and testnet addresses should be distinguishable.
- Invalid addresses must be rejected.
- Address derivation must be deterministic where the selected wallet standard requires it.
- Private key material must never be recoverable from a public address.

The address format remains TO BE FINALIZED.

---

16. MEMORY AND APPLICATION SECURITY

Wallet software must minimize exposure of secrets in memory.

Requirements:

- Avoid unnecessary copies of secret data.
- Clear sensitive buffers when practical.
- Avoid serializing secrets into debug output.
- Avoid crash reports containing wallet secrets.
- Avoid clipboard copying of recovery phrases without explicit user action.
- Warn users about clipboard exposure.
- Lock the wallet after configured inactivity.

Complete memory protection cannot be guaranteed by ordinary application code.

The implementation must document platform-specific limitations.

---

17. NETWORK SECURITY

Wallet-to-node communication must use appropriate network protections.

Requirements:

- Validate received blockchain data.
- Treat remote node responses as untrusted.
- Use HTTPS for remote production APIs.
- Prevent accidental connection to the wrong network.

- Never transmit private keys or recovery phrases.
- Verify transaction details independently before signing.

A remote node may provide incorrect or incomplete information.

Wallets should use suitable verification methods rather than blindly trusting a server's reported balance.

---

18. SECURITY TESTING

Required wallet security tests:

Key Management

- Randomness quality tests
- Key derivation tests
- Invalid key rejection
- Secret exposure tests

Encryption

- Encryption/decryption tests
- Wrong-password tests
- Corrupted wallet-file tests
- Backup compatibility tests

Recovery

- Recovery phrase validation
- Incorrect phrase rejection
- Wallet restoration
- Address consistency
- Balance reconstruction

Transaction Signing

- Valid signature tests
- Invalid signature tests
- Incorrect network tests
- Modified transaction tests
- Recipient substitution tests

Application Security

- Sensitive logging tests
- API permission tests

- Local storage inspection
- Frontend/backend isolation tests
- Android backup configuration tests
- Windows local file protection tests

---

19. SECURITY INCIDENT RESPONSE

If a wallet vulnerability is discovered:

1. Assess the vulnerability.
2. Determine affected versions.
3. Determine whether private keys may be exposed.
4. Stop distribution of affected releases if necessary.
5. Publish a security advisory.
6. Prepare a verified fix.
7. Notify affected users.
8. Provide migration guidance when required.
9. Review the cause.
10. Add regression tests.

Users must be advised to move funds to a newly secured wallet if private keys may have been compromised.

---

20. ITEMS REQUIRING FINAL DECISIONS

The following must be finalized before wallet implementation:

1. Cryptographic key algorithm
2. Signature algorithm
3. Seed phrase standard
4. Entropy size
5. Key derivation scheme
6. Address derivation path
7. Address encoding
8. Wallet encryption algorithm
9. Password derivation function
10. Backup file format
11. Recovery format
12. Transaction signing serialization
13. Android secure storage design
14. Windows secure storage design

All cryptographic decisions must be reviewed before production use.

---

21. FINAL WALLET SECURITY PRINCIPLES

1. Users retain control of their funds.
2. Private keys remain local.
3. Recovery phrases are never transmitted to servers.
4. Wallet data is encrypted at rest.
5. Transaction signing occurs locally.
6. Sensitive information is excluded from logs.
7. Backups are recoverable and protected.
8. Android and Windows wallets follow the same security model.
9. Cryptographic standards must be reviewed before adoption.
10. Security testing is mandatory before public release.

DOCUMENT STATUS

Document: Wallet_Security_Design.md
Version: 1.0
Project: SAECOIN
Wallet Platforms: Android and Windows
Key Algorithm: Not Finalized
Mnemonic Standard: Not Finalized
Encryption Design: Not Finalized
Address Format: Not Finalized
Implementation Status: Not Started